

Projeto: ForkUp Clean Arch

Grupo 74:

Nome: Luís Felipe Garcia Menezes - **RM:** 371055

1. Introdução

Descrição do Problema

Um grupo de restaurantes da região decidiu desenvolver um sistema de gestão compartilhado para reduzir custos com soluções individuais. O objetivo é criar uma plataforma única que permita aos restaurantes gerenciar suas operações e aos clientes consultar informações, avaliar estabelecimentos e realizar pedidos online. Devido a limitações financeiras, o sistema será desenvolvido em fases, possibilitando uma implementação gradual, com melhorias contínuas baseadas no uso e feedback dos usuários.

Objetivo do Projeto

Desenvolver uma API REST robusta, escalável e de fácil manutenção utilizando Spring Boot e os princípios da Clean Architecture. O projeto tem como objetivo demonstrar a aplicação de uma arquitetura desacoplada, onde as regras de negócio permanecem independentes de frameworks, banco de dados e demais detalhes de infraestrutura.

A aplicação realiza o gerenciamento de usuários, endereços, restaurantes e cardápio, adotando boas práticas de desenvolvimento, como validação das regras de negócio, criptografia de senhas, inversão de dependências, persistência em banco de dados relacional, testes automatizados. Sua estrutura foi projetada para facilitar a manutenção, a evolução do sistema e a substituição de tecnologias sem impactar a camada de domínio.

2. Arquitetura do Sistema

A aplicação **ForkUp** foi desenvolvida seguindo os princípios da **Clean Architecture**, com foco na separação de responsabilidades, baixo acoplamento, alta coesão e independência de frameworks e tecnologias de infraestrutura. Essa abordagem mantém as regras de negócio no centro da aplicação, permitindo que detalhes como banco de dados, APIs e bibliotecas possam ser alterados sem impactar o domínio.

A solução é executada em ambiente containerizado utilizando Docker, com dois serviços principais: a aplicação backend (Spring Boot) e o banco de dados PostgreSQL, que se comunicam através de uma rede interna.

O fluxo de uma requisição segue o padrão:

Cliente → Controller → Use Case → Gateway (Interface) → Gateway Implementation → Banco de Dados

A resposta percorre o caminho inverso, retornando da camada de infraestrutura para o caso de uso, onde os dados são convertidos em DTOs antes de serem enviados ao cliente.

Dessa forma, a camada de domínio permanece isolada dos detalhes de implementação, tornando a aplicação mais testável, flexível e preparada para futuras evoluções.

2.1 Estrutura da Aplicação

Controller REST

É a porta de entrada da aplicação.

- Responsável por receber requisições HTTP e atuar como adaptador entre o cliente e a aplicação.
- Realiza validações básicas de entrada com Jakarta Validation (@Valid).
- Invoca o caso de uso correspondente.
- Converte o resultado em DTO de resposta.

O Controller não contém regras de negócio.

Use Cases

Representam as funcionalidades da aplicação. Cada Use Case é responsável por executar um único caso de uso, como por exemplo criar, buscar, atualizar ou remover um usuário.

- Orquestrar o fluxo da operação.
- Aplicar regras de negócio.
- Manipular entidades de domínio.
- Utilizar Gateways para acessar dados.
- Retornar o resultado para a camada de apresentação.

Os Use Cases conhecem apenas interfaces (Gateways), nunca implementações.

Domain

É o núcleo da aplicação. Contém:

- Entidades de domínio.
- Regras de negócio referente ao domínio.
- Comportamentos das entidades.

Essa camada não possui dependência de Spring, JPA ou qualquer tecnologia de infraestrutura.

Gateways

São interfaces que definem como os casos de uso interagem com recursos externos. Exemplos:

- UsuarioGateway
- EnderecoGateway
- TipoUsuarioGateway

Os Gateways descrevem **o que precisa ser feito**, mas não **como será feito**.

Infrastructure

Contém as implementações dos Gateways. É responsável por:

- Persistência no banco de dados.
- Implementação utilizando Spring Data JPA.
- Conversão entre entidades de domínio e entidades JPA.

- Integrações externas.

Essa camada depende do Spring e conhece os detalhes da infraestrutura.

Entidades de Persistência

Representam o modelo utilizado pelo banco de dados. São anotadas com:

- @Entity
- @Table
- Relacionamentos JPA

Essas entidades existem apenas na camada de infraestrutura e não fazem parte do domínio.

Mappers

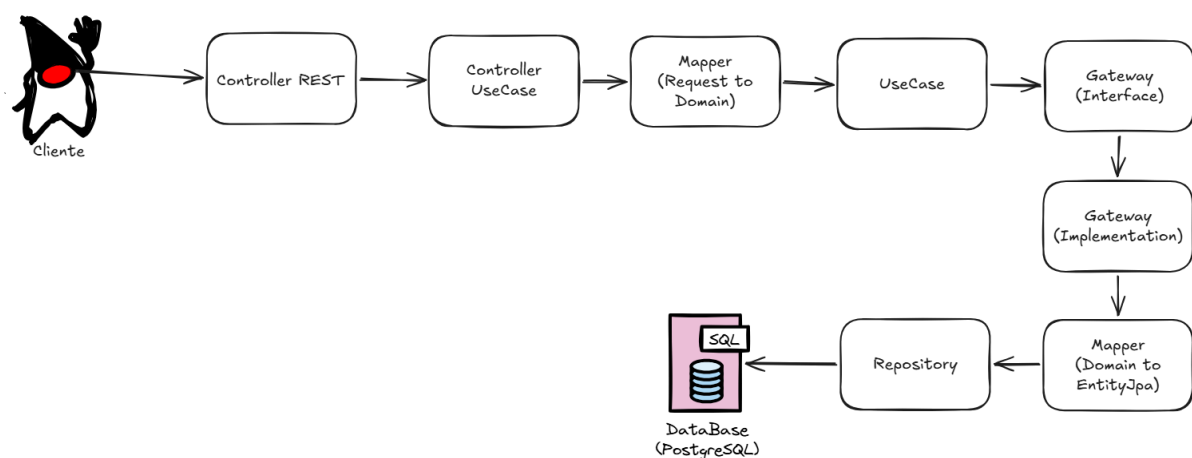
Realizam a conversão entre:

- DTO $\rightleftarrows$ Domínio
- Domínio $\rightleftarrows$ Entity JPA

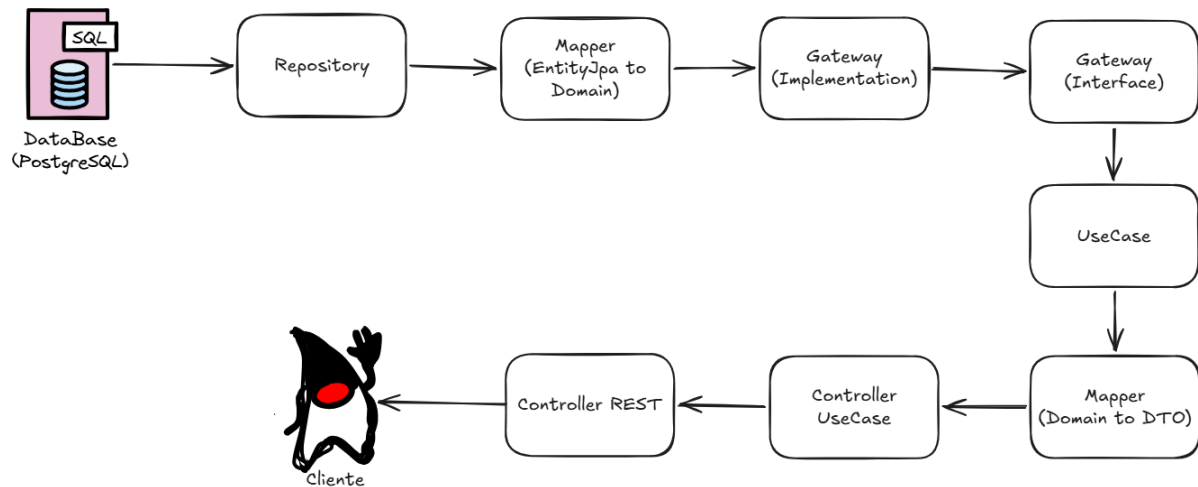
Dessa forma, cada camada trabalha apenas com seus próprios modelos.

Fluxo da Requisição

Entrada



Saida



Obs.: O **Controller REST** pertence à camada de apresentação e lida com o protocolo HTTP, enquanto o **Controller do Use Case** pertence à camada de aplicação e coordena a execução dos casos de uso, mantendo-os independentes da infraestrutura.

2.2 Containerização

A aplicação é executada em um ambiente containerizado utilizando **Docker Compose**, permitindo que todos os serviços necessários sejam iniciados de forma integrada e padronizada.

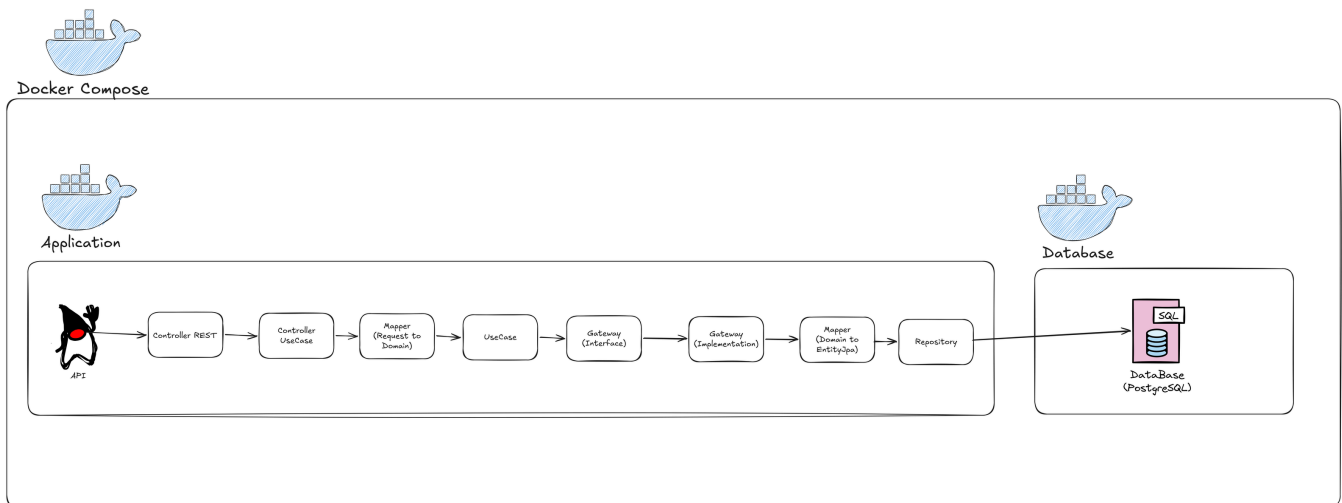
O ambiente é composto por dois serviços principais:

- **app** → Responsável por executar a aplicação Spring Boot.
- **postgres** → Responsável pela persistência dos dados da aplicação.

Os containers se comunicam por meio de uma rede interna do Docker, utilizando o nome do serviço como *hostname* (por exemplo, postgres), eliminando a necessidade de configurações manuais de rede.

As configurações do ambiente, como credenciais do banco de dados, portas e demais parâmetros, são externalizadas por meio de variáveis de ambiente definidas no arquivo .env, facilitando a configuração da aplicação em diferentes ambientes sem necessidade de alterar o código-fonte.

2.3 Diagrama da Arquitetura



3. Modelagem do Domínio e Relacionamentos

O domínio da aplicação foi modelado com foco na representação das regras de negócio, mantendo independência em relação aos detalhes de persistência. As entidades descrevem os principais conceitos do sistema e seus relacionamentos, enquanto a camada de infraestrutura é responsável pelo mapeamento dessas informações para o banco de dados.

3.1 Entidades do Domínio

- **Tipo Usuário**
 - Representa a classificação de um usuário dentro do sistema.
 - Contém informações como descrição do tipo de usuário e data de criação
 - O tipo de usuário é utilizado para definir regras e permissões específicas da aplicação

- **Endereço**
 - Representa o endereço associado a um usuário ou restaurante
 - Contém dados como rua, número, cidade, estado e CEP
 - Essa entidade é reutilizada por diferentes agregados do domínio.
- **Usuário**
 - Representa uma pessoa cadastrada no sistema
 - Contém informações como nome, email, login e senha
 - A senha é armazenada de forma criptografada
- **Restaurante**
 - Representa um restaurante cadastrado na plataforma
 - Contém informações como nome, tipo da Cozinha, Proprietario e cardapio
 - Além dos seus dados cadastrais, mantém a relação com os itens disponíveis para venda.
- **Item do Cardápio**
 - Representa um produto oferecido por um restaurante.
 - Contém informações como nome, descrição, preço, disponibilidade consumo e caminho da imagem do produto
 - Cada item pertence ao cardápio de um único restaurante.

3.2 Relacionamentos

- Um **Tipo de Usuário** pode estar associado a vários **Usuários**.
- Um **Usuário** possui um único **Endereço**.
- Um **Restaurante** possui um único **Endereço**.
- Um **Restaurante** possui um único **Dono (Usuário)**
- Um **Restaurante** possui um ou mais **Itens de Cardápio**.
- Cada **Item de Cardápio** pertence a um único **Restaurante**.

3.3 Regras de negócio refletidas

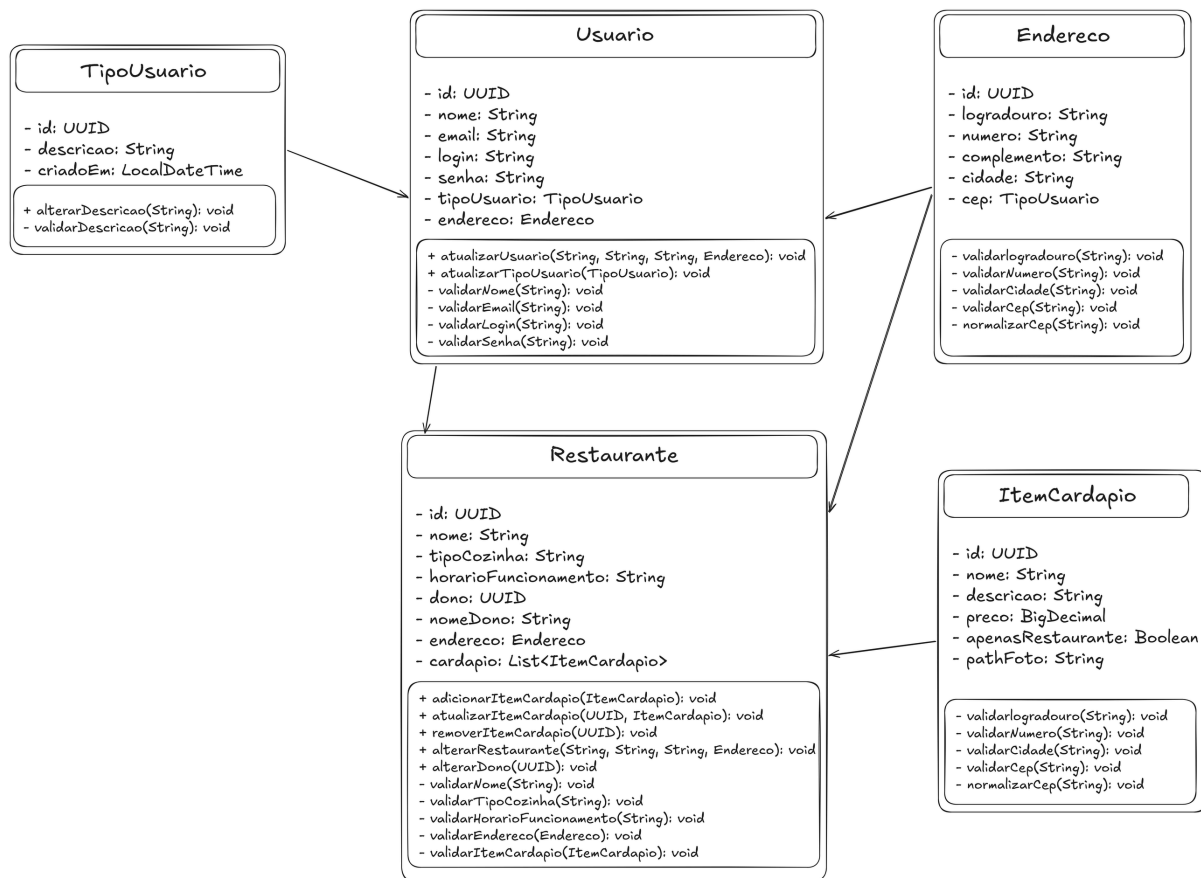
Entre as principais regras implementadas no domínio estão:

- Não é permitido cadastrar usuários com e-mail ou login duplicados.
- O e-mail deve possuir um formato válido.
- A senha deve atender aos critérios mínimos de segurança definidos pela aplicação.
- Todo usuário deve possuir um tipo de usuário válido.

- Todo restaurante deve possuir um endereço associado.
- Os itens do cardápio pertencem exclusivamente a um restaurante.
- Só é permitido a troca de dono do Restaurante caso o novo Dono não esteja vinculado a um restaurante.

3.4 Diagramas

Abaixo está o diagrama de entidades e relacionamentos do sistema:



4. Descrição dos Endpoints

4.1 Tabela de Endpoints

Endpoint	Método	Descrição
/api/tipo-usuario	GET	Lista tipos de usuário
/api/tipo-usuario/{id}	GET	Busca tipo de usuário por ID
/api/tipo-usuario	POST	Cria um novo tipo de usuário
/api/tipo-usuario/{id}	PUT	Atualiza a descricao do tipo de usuário
/api/tipo-usuario/{id}	DELETE	Remove tipo de usuario
/api/usuario	GET	Lista usuários com paginação
/api/usuario/{id}	GET	Busca usuário por ID
/api/usuario	POST	Cria um novo usuário
/api/usuario/{id}	PUT	Atualiza dados do usuário
/api/usuario/alterar-tipo/{id}	PUT	Altera o tipo do usuário
/api/usuario/{id}	DELETE	Remove usuário
/api/restaurante	GET	Lista restaurantes com paginação
/api/restaurante/{id}	GET	Busca restaurante por ID
/api/restaurante	POST	Cria um novo restaurante

/api/restaurante/{id}	PUT	Atualiza dados do restaurante
/api/restaurante/alterar-dono/{id}	PUT	Alterar dono do restaurante
/api/restaurante	DELETE	Remove restaurante
/api/restaurante/{idRestaurante}/cardapio	GET	Lista os itens do cardápio
/api/restaurante/{idRestaurante}/cardapio	POST	Cria um novo item no cardápio
/api/restaurante/{idRestaurante}/cardapio/{itemId}	PUT	Atualiza dados do item do cardápio de um restaurante
/api/restaurante/{idRestaurante}/cardapio/{itemId}	DELETE	Remove item do cardápio de um restaurante

4.2 Exemplos de Requisição e resposta

Tipo Usuário

Criar Tipo Usuário

POST - /api/tipo-usuario

Request:

```
{
  "descricao" : "Suporte"
}
```

Reponse (200):

"00a829a6-d362-4334-86ed-c10ef6456cdc"

Usuários

Buscar usuário por ID

GET - /api/usuario/{id}

Response (200):

```
{
  "id": "809e9b65-d878-4bef-81bd-74379bf24eb3",
  "nome": "Maria Oliveira",
  "email": "maria.oliveira@example.com",
  "login": "maria.oliveira",
  "tipoUsuario": {
    "id": "2f8c9a0b-2d3e-4b6c-9a0b-2d3e4b6c9a0b",
    "descricao": "Dono de Restaurante",
  },
  "endereco": {
    "id": "fb592482-462b-4902-a0f7-0b658374d28d",
    "logradouro": "Avenida Teste",
    "numero": "456",
    "complemento": "AP 1401",
    "cidade": "Cidade Teste",
    "cep": "06128674",
  }
}
```

5. Documentação Swagger

5.1 Documentação da API com Swagger

A API do **ForkUp** possui documentação interativa gerada automaticamente utilizando o **SpringDoc OpenAPI (Swagger)**, permitindo visualizar e testar os endpoints disponíveis de forma prática, sem a necessidade de ferramentas externas.

Após iniciar a aplicação, a documentação pode ser acessada em:

<http://localhost:3710/swagger-ui.html>

Por meio da interface do Swagger é possível:

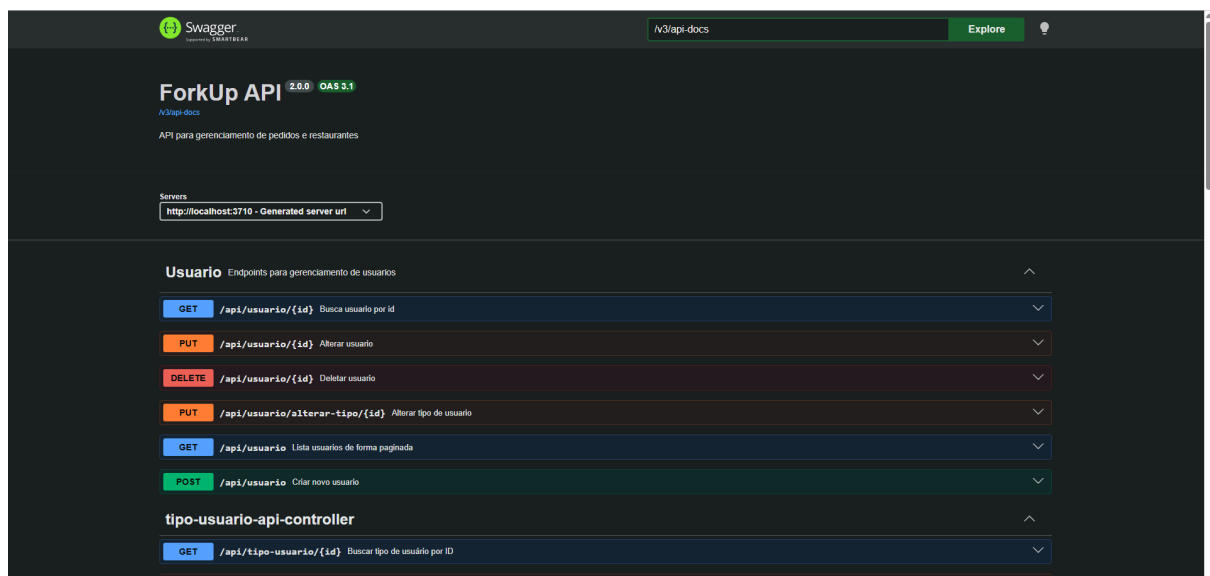
- Visualizar todos os endpoints disponibilizados pela API
- Consultar detalhes de cada operação (GET, POST, PUT, DELETE, etc.)
- Ver os parâmetros de entrada e os modelos de requisição e resposta
- Identificar os códigos de status HTTP retornados por cada endpoint
- Executar requisições diretamente pela interface utilizando o recurso **"Try it out"**

Como a documentação é gerada automaticamente a partir do código da aplicação, ela permanece sincronizada com os endpoints implementados, facilitando o desenvolvimento, os testes e a integração com outros sistemas.

5.2 Exemplos de Interface

Tela Inicial

A tela inicial apresenta todos os controllers e endpoints organizados por categoria.

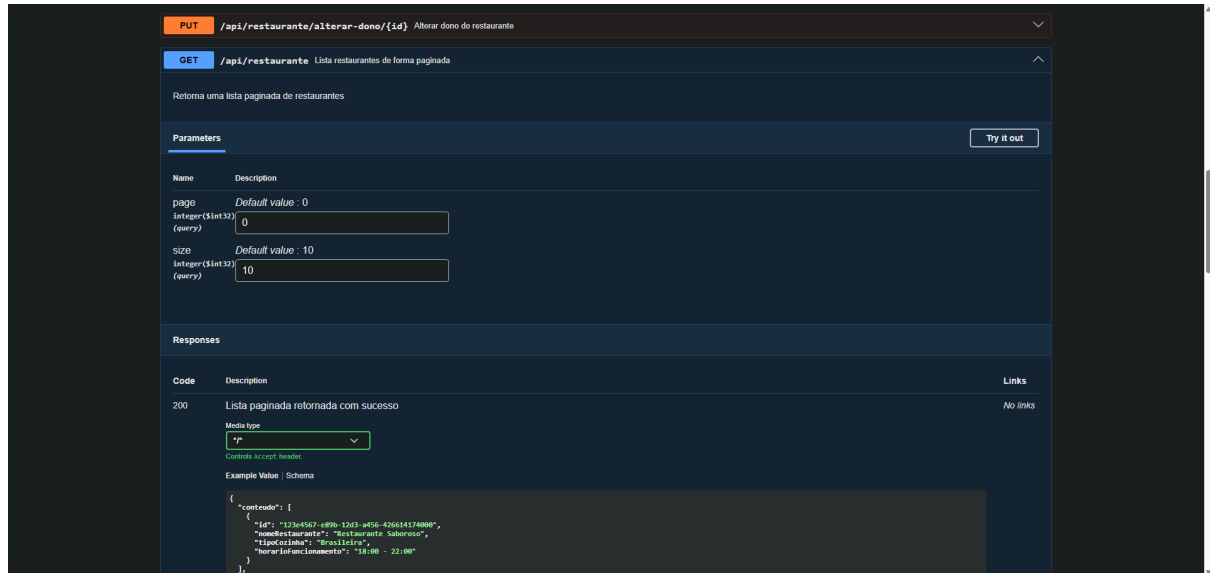


tipo-usuario-api-controller	
GET	/api/tipo-usuario/{id} Buscar tipo de usuário por ID
PUT	/api/tipo-usuario/{id} Alterar descrição do tipo de usuário
DELETE	/api/tipo-usuario/{id} Deletar tipo de usuário
GET	/api/tipo-usuario Lista todos os tipos de usuário
POST	/api/tipo-usuario Criar novo tipo de usuário
restaurante-api-controller	
GET	/api/restaurante/{id} Busca restaurante por id
PUT	/api/restaurante/{id} Alterar restaurante
DELETE	/api/restaurante/{id} Deletar restaurante
PUT	/api/restaurante/{idRestaurante}/cardapio/{itemId} Atualizar item do cardápio
PUT	/api/restaurante/alterar-dono/{id} Alterar dono do restaurante
GET	/api/restaurante Lista restaurantes de forma paginada
POST	/api/restaurante Criar novo restaurante
GET	/api/restaurante/{idRestaurante}/cardapio Listar cardápio do restaurante
POST	/api/restaurante/{idRestaurante}/cardapio Adicionar item ao cardápio
DELETE	/api/restaurante/{idRestaurante}/cardapio/{idItemCardapio} Remover item do cardápio

POST	/api/restaurante/{idRestaurante}/cardapio/{idItemCardapio} Adicionar item ao cardápio
DELETE	/api/restaurante/{idRestaurante}/cardapio/{idItemCardapio} Remover item do cardápio
Schemas	
EnderecoVO > Expand all object	
UsuarioUpdateVO > Expand all object	
EnderecoResponse > Expand all object	
TipoUsuarioResponse > Expand all object	
UsuarioResponseFull > Expand all object	
ErrorResponse > Expand all object	
UsuarioAlterarTipoVO > Expand all object	
TipoUsuarioVO > Expand all object	
RestauranteUpdateVO > Expand all object	
RestauranteResponseFull > Expand all object	
ItemCardapioVO > Expand all object	

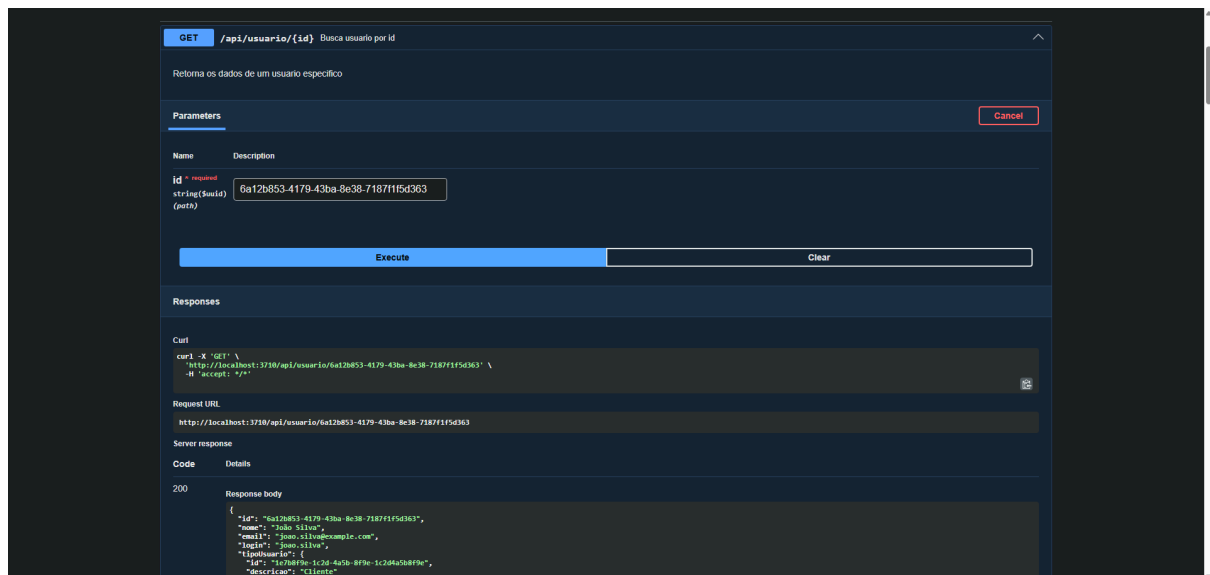
Exemplo de Endpoint Expandido

Ao expandir um endpoint, é possível visualizar detalhes da requisição, incluindo parâmetros, corpo da requisição e exemplos de resposta.



Execução de requisição

A funcionalidade "Try it out" permite testar os endpoints diretamente pela interface.



6. Documentação Postman

6.1 Testes da API com Postman

Além da documentação interativa disponibilizada pelo Swagger, o projeto fornece uma **Collection do Postman** com requisições pré-configuradas para facilitar os testes e a validação dos principais fluxos da API.

A collection contempla os endpoints implementados na aplicação, permitindo executar operações de cadastro, consulta, atualização e remoção de recursos de forma rápida e organizada.

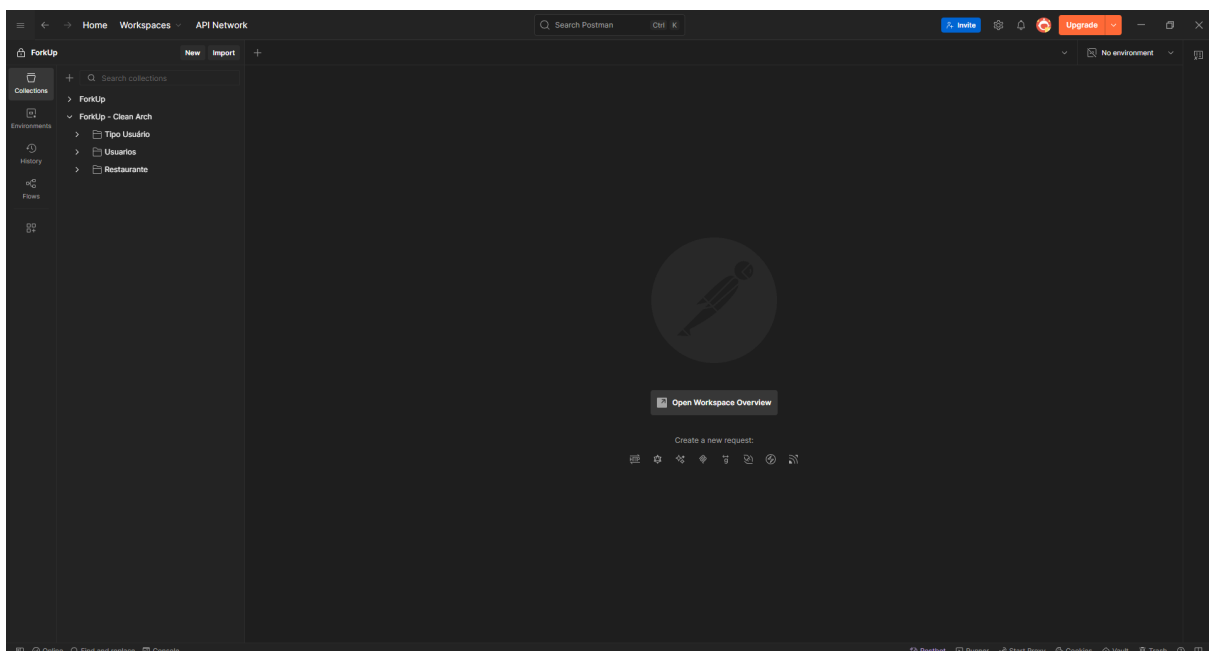
Arquivo da Collection

- Nome: ForkUp - Clean Arch.postman_collection.json
- Localização: raiz do projeto

6.2 Exemplos da Interface

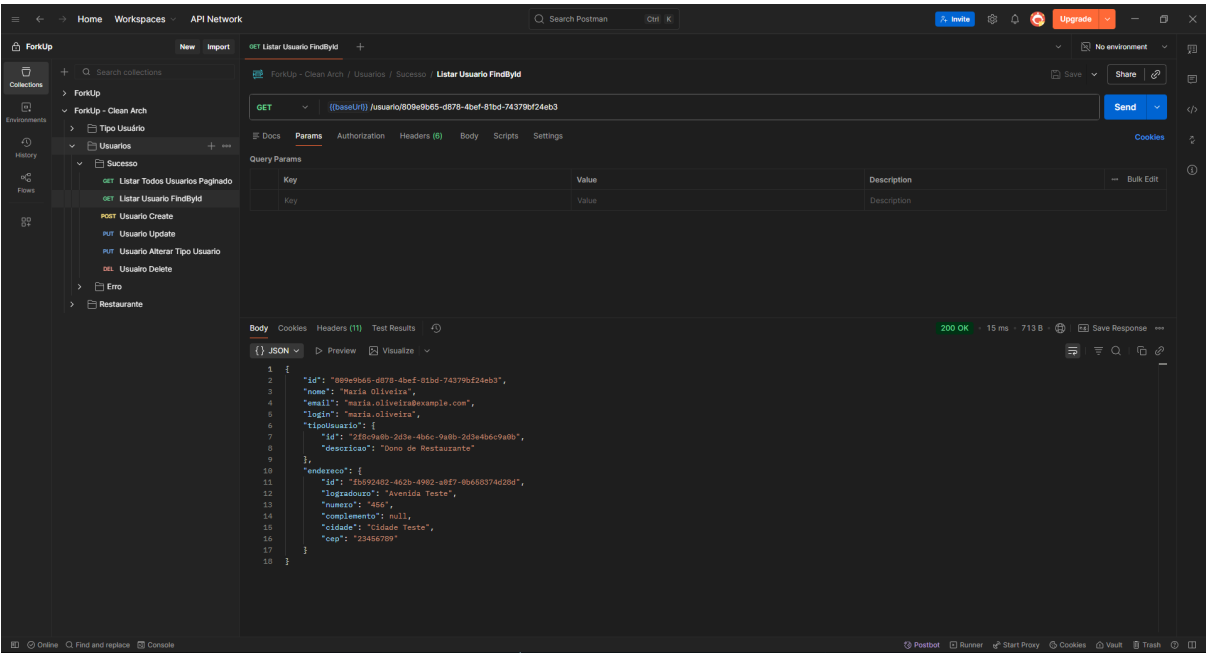
Collection importada

Visualização da collection com todos os endpoints organizados por categoria.



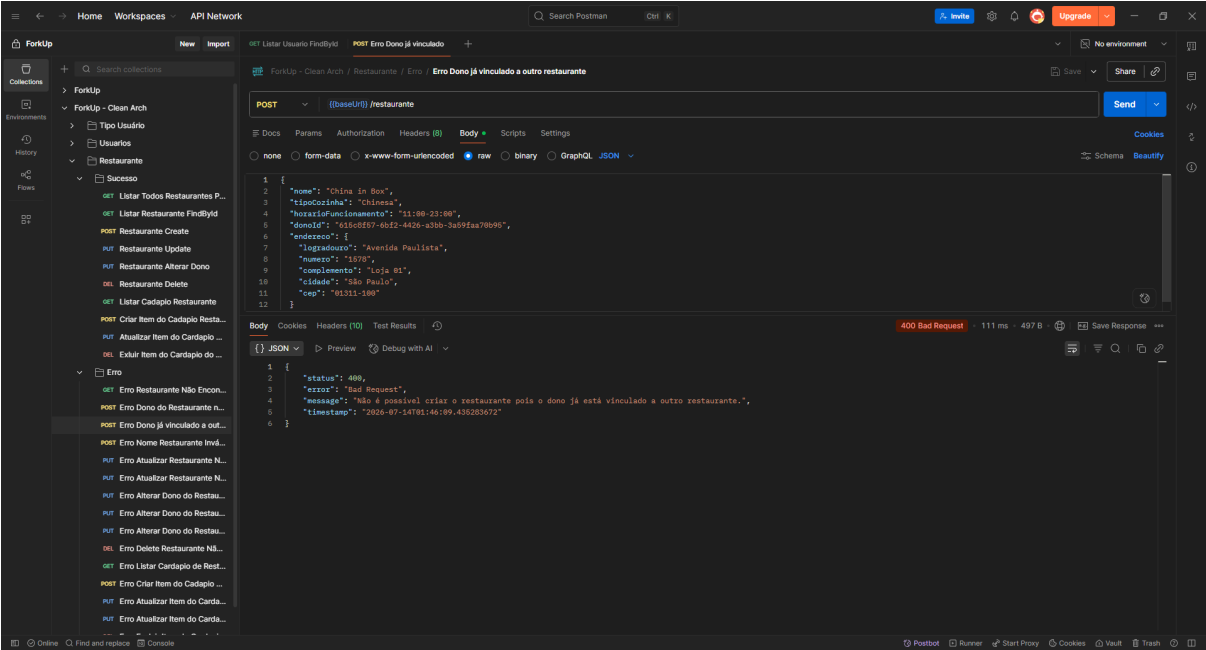
Execução de requisição

Exemplo de execução de uma requisição com retorno da API.



Separação em Sucesso e Error

As requisições estão organizadas por domínio (Tipo Usuário, Usuário e Restaurante) e para cada um possuem pastas, com massas de testes prontas para as execuções de sucesso e error

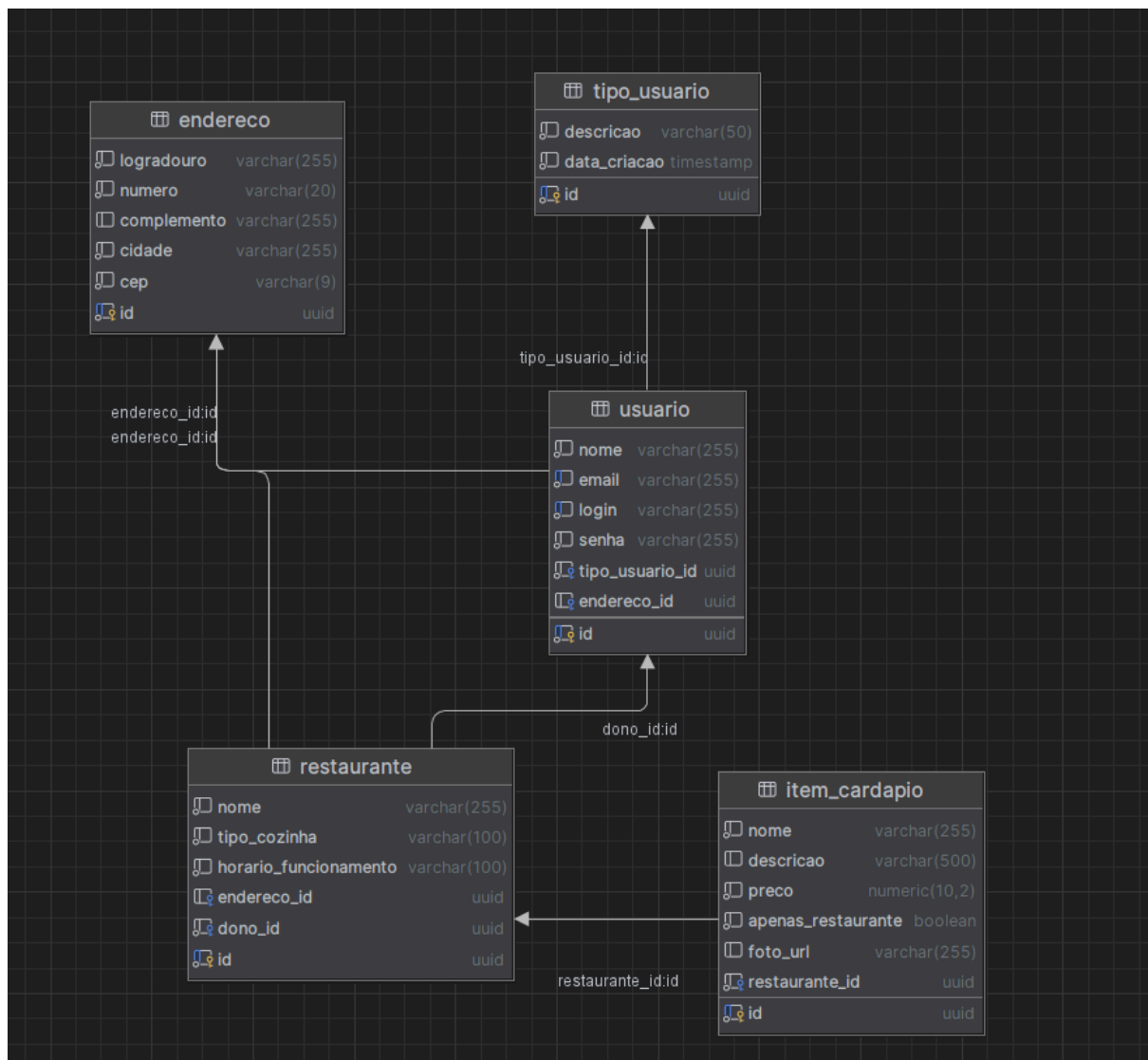


7. Estrutura do Banco de Dados

A persistência de dados da aplicação **ForkUp** é realizada em um banco de dados relacional **PostgreSQL**, cuja estrutura foi projetada para garantir a integridade, a consistência e o suporte às regras de negócio implementadas no domínio da aplicação.

Embora a **Clean Architecture** mantenha o domínio desacoplado dos detalhes de persistência, a camada de infraestrutura é responsável pelo mapeamento das entidades de domínio para o modelo relacional, utilizando o **Spring Data JPA** como mecanismo de acesso aos dados.

Atualmente, o sistema é composto pelas seguintes tabelas:



8. Testes Automatizados

O projeto possui uma estratégia de testes automatizados para garantir a confiabilidade das regras de negócio e o correto funcionamento da aplicação. Os testes foram desenvolvidos seguindo a estrutura da **Clean Architecture**, validando tanto os casos de uso de forma isolada quanto a integração entre as camadas da aplicação.

8.1 Testes Unitários

Os testes unitários são responsáveis por validar as regras de negócio implementadas nos **Use Cases** e nas entidades de domínio, executando cada caso de uso de forma isolada.

Para isso, as dependências externas, como os **Gateways**, são simuladas utilizando **Mockito**, permitindo que apenas a lógica de negócio seja testada, sem necessidade de acesso ao banco de dados ou à infraestrutura da aplicação.

Esses testes garantem que os casos de uso se comportem corretamente diante de diferentes cenários, incluindo validações, exceções e fluxos de sucesso.

Exemplo de teste unitário:

```
ListarCardapioRestauranteUseCaseTest.java
21
22 @ExtendWith(MockitoExtension.class) @LuisFelipeGM
23 @DisplayName("Testes unitários para o caso de uso ListarCardapioRestauranteUseCase")
24 public class ListarCardapioRestauranteUseCaseTest { LuisFelipeGM, 09/07/2026 22:31 • test: adição de testes unitários para domain e useCases
25
26     @Mock 7 usages
27     private RestauranteGateway restauranteGateway;
28
29     @InjectMocks 2 usages
30     private ListarCardapioRestauranteUseCase listarCardapioRestauranteUseCase;
31
32     @Test @LuisFelipeGM
33     @DisplayName("Deve listar o cardápio de um Restaurante com sucesso")
34     void deveListarCardapioComSucesso() {
35         UUID idRestaurante = UUID.randomUUID();
36         List<ItemCardapio> cardapio = List.of(criarItemCardapio());
37
38         when(restauranteGateway.existsRestaurante(idRestaurante)).thenReturn(true);
39         when(restauranteGateway.listarCardapio(idRestaurante)).thenReturn(cardapio);
40
41         List<ItemCardapio> retorno = listarCardapioRestauranteUseCase.execute(idRestaurante);
42
43         assertNotNull(retorno);
44         assertEquals(cardapio.size(), retorno.size());
45         assertEquals(cardapio.getFirst().getNome(), retorno.getFirst().getNome());
46         verify(restauranteGateway, times(wantedNumberOfInvocations: 1)).existsRestaurante(idRestaurante);
47         verify(restauranteGateway, times(wantedNumberOfInvocations: 1)).listarCardapio(idRestaurante);
48     }
}
```

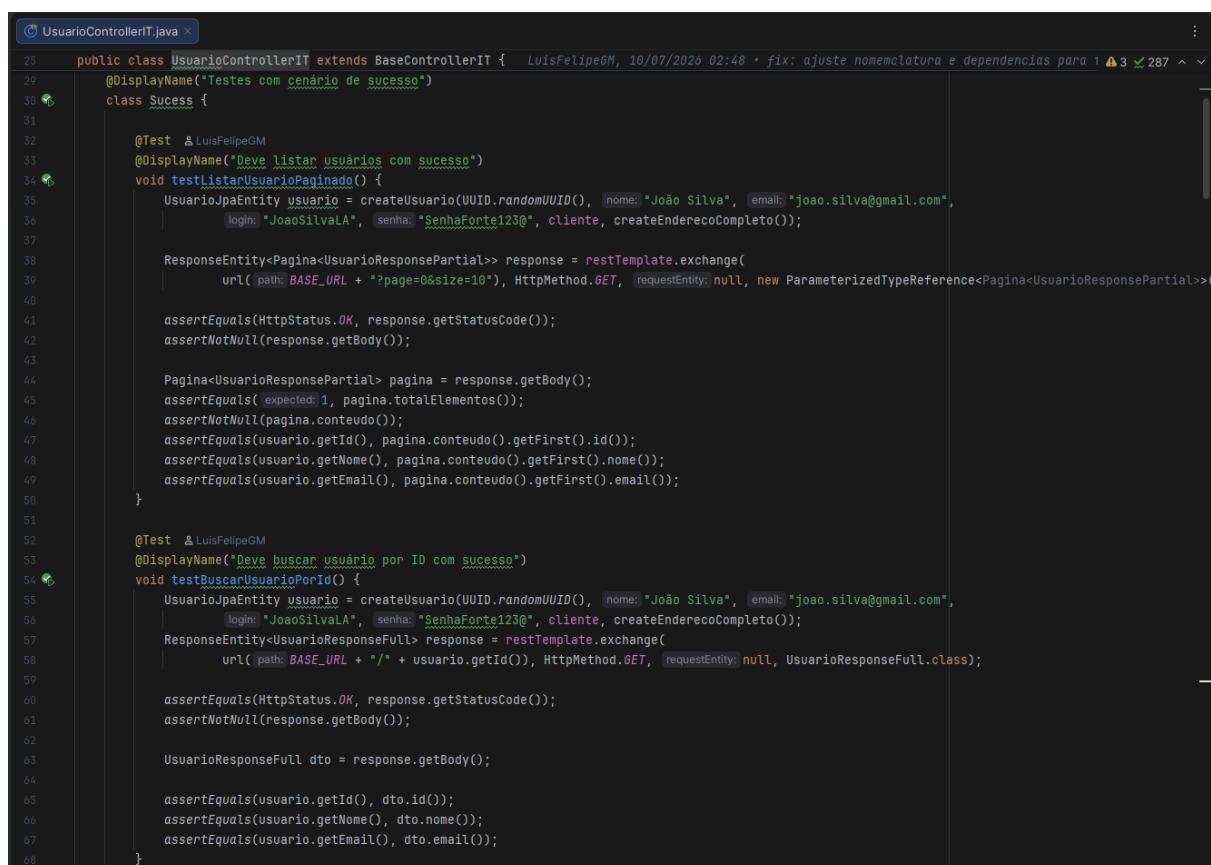
8.2 Testes de Integração

Os testes de integração verificam o funcionamento conjunto das camadas da aplicação, garantindo que a comunicação entre Controllers, Use Cases, Gateways e persistência ocorra conforme esperado.

Para proporcionar um ambiente próximo ao de produção, os testes utilizam um banco de dados **PostgreSQL** executado por meio do **Testcontainers**, permitindo que cada execução seja realizada em um ambiente isolado e descartável.

Dessa forma, é possível validar não apenas as regras de negócio, mas também o comportamento da persistência, dos mapeamentos, das consultas e das transações realizadas pela aplicação.

Exemplo de teste de integração:



```
25 public class UsuarioControllerIT extends BaseControllerIT {
26     @DisplayName("Testes com cenário de sucesso")
27     class Success {
28
29         @Test & LuisFelipeGM
30         @DisplayName("Deve listar usuários com sucesso")
31         void testListarUsuarioPaginado() {
32             UsuarioJpaEntity usuario = createUsuario(UUID.randomUUID(), nome: "João Silva", email: "joao.silva@gmail.com",
33                 login: "JoaoSilvaLA", senha: "SenhaForte123@", cliente, createEnderecoCompleto());
34
35             ResponseEntity<Pagina<UsuarioResponsePartial>> response = restTemplate.exchange(
36                 url(path: BASE_URL + "?page=0&size=10"), HttpMethod.GET, requestEntity: null, new ParameterizedTypeReference<Pagina<UsuarioResponsePartial>>() {
37
38                 });
39
40             assertEquals(HttpStatus.OK, response.getStatusCode());
41             assertNotNull(response.getBody());
42
43             Pagina<UsuarioResponsePartial> pagina = response.getBody();
44             assertEquals(expected: 1, pagina.totalElementos());
45             assertNotNull(pagina.conteudo());
46             assertEquals(usuario.getId(), pagina.conteudo().getFirst().id());
47             assertEquals(usuario.getNome(), pagina.conteudo().getFirst().nome());
48             assertEquals(usuario.getEmail(), pagina.conteudo().getFirst().email());
49         }
50     }
51
52     @Test & LuisFelipeGM
53     @DisplayName("Deve buscar usuário por ID com sucesso")
54     void testBuscarUsuarioPorId() {
55         UsuarioJpaEntity usuario = createUsuario(UUID.randomUUID(), nome: "João Silva", email: "joao.silva@gmail.com",
56             login: "JoaoSilvaLA", senha: "SenhaForte123@", cliente, createEnderecoCompleto());
57
58         ResponseEntity<UsuarioResponseFull> response = restTemplate.exchange(
59             url(path: BASE_URL + "/" + usuario.getId()), HttpMethod.GET, requestEntity: null, UsuarioResponseFull.class);
60
61         assertEquals(HttpStatus.OK, response.getStatusCode());
62         assertNotNull(response.getBody());
63
64         UsuarioResponseFull dto = response.getBody();
65
66         assertEquals(usuario.getId(), dto.id());
67         assertEquals(usuario.getNome(), dto.nome());
68         assertEquals(usuario.getEmail(), dto.email());
69     }
69 }
```

8.3 Tecnologias Utilizadas

Os testes foram desenvolvidos utilizando as seguintes ferramentas:

- **JUnit 5** para execução da suíte de testes.
- **Mockito** para criação de mocks e isolamento das dependências nos testes unitários.
- **Spring Boot Test** para inicialização do contexto da aplicação nos testes de integração.
- **Testcontainers** para criação de um ambiente temporário com PostgreSQL durante a execução dos testes.
- **PostgreSQL** como banco de dados utilizado nos testes de integração.

8.4 Execução dos Testes

Todos os testes podem ser executados por meio do Gradle com o comando:

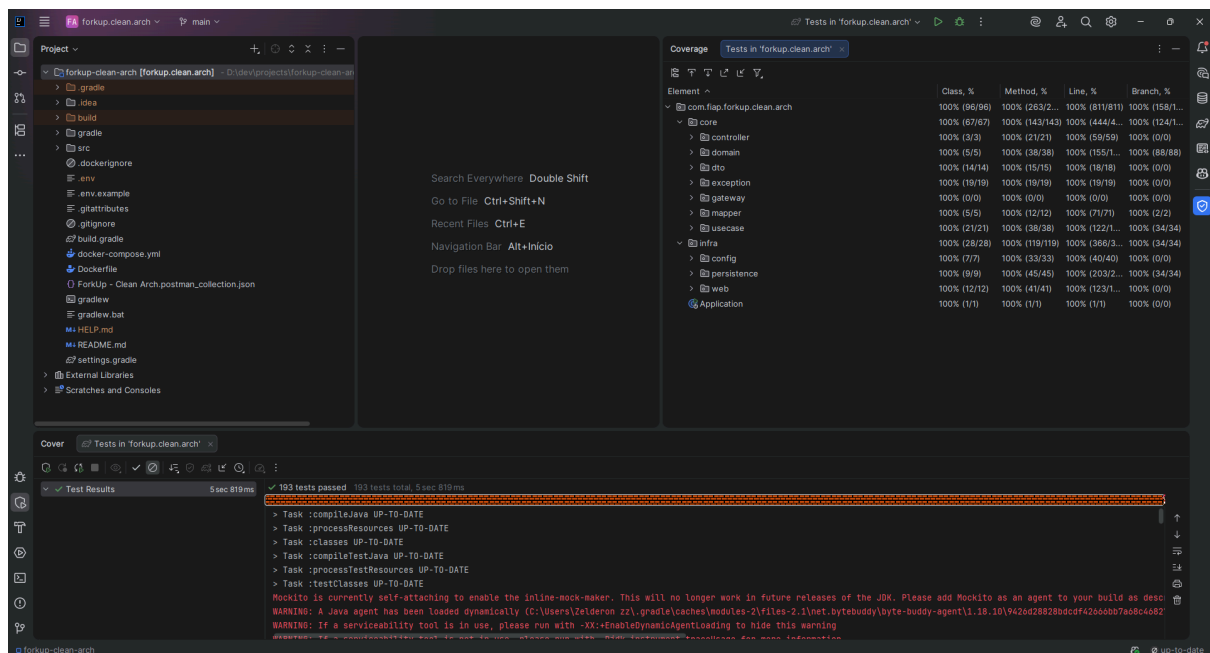
`./gradlew test`

Ao final da execução, o Gradle gera um relatório contendo o resultado de todos os testes, permitindo identificar rapidamente possíveis falhas e garantir a qualidade da aplicação antes de novas alterações ou implantações.

8.5 Cobertura de Testes

Durante o desenvolvimento, foi utilizada a funcionalidade **Code Coverage** do **IntelliJ IDEA** para analisar o percentual de código exercitado pelos testes automatizados.

A cobertura permite identificar classes, métodos e linhas de código executadas durante os testes, auxiliando na validação das regras de negócio e na identificação de trechos que podem receber novos cenários de teste. Aplicação com cobertura de testes em 100%:



9. Execução da Aplicação com Docker Compose

9.1 Clonar o repositório

```
git clone https://github.com/LuisFelipeGM/ForkUp-Clean-Architecture.git
cd ForkUp-Clean-Architecture
```

9.2 Configurar variáveis de ambiente

O projeto utiliza variáveis de ambiente para configuração do banco de dados e da aplicação. Criar arquivo .env:

Copie o arquivo de exemplo:

```
cp .env.example .env
```

No Windows (PowerShell):

```
copy .env.example .env
```

Editar o .env

```
DB_NAME=forkup_db
```

```
DB_USER=admin
```

```
DB_PASSWORD=sua_senha
```

```
DB_PORT=5432
```

```
APP_PORT=3710
```

Explicação das variáveis:

Variável	Descrição
DB_NAME	Nome do banco de dados

DB_USER	Usuário do banco
DB_PASSWORD	Senha do banco
DB_PORT	Porta do PostgreSQL
APP_PORT	Porta da aplicação

9.3 Executar a aplicação

Para construir a imagem e subir os containers:

docker compose up --build

9.4 Acessar a aplicação

Após a inicialização:

- API: <http://localhost:3710>
- Swagger: <http://localhost:3710/swagger-ui.html>

Observações importantes

- O arquivo .env não é versionado por conter dados sensíveis
- Utilize o .env.example como base
- A aplicação se conecta ao banco utilizando o nome do serviço (postgres) como host

10. Repositório do Projeto

URL do Repositório:

<https://github.com/LuisFelipeGM/ForkUp-Clean-Architecture>

Vídeo de apresentação:

 Tech Challenge - Fase 02 - ForkUp - Apresentação de Funcionalidades Solicit...